

Receipt upload, taxes and auto-itemize

Architecture and design review document

A Go HTTP service. A user uploads a receipt, and processing creates **exactly one transaction** with its **tax lines as separate records**, **auto-itemized line items**, the **raw OCR text**, and an **itemize status** (`COMPLETE` , `NEEDS_REVIEW` or `FAILED`). Users can edit, merge and split items. An edit that breaks the totals is rejected with `409` and the mismatch, and the service never invents a balancing line or adjusts a total.

OCR is stubbed, as the brief allows. No vendor is called and no API key exists. A text upload is its own OCR output, and an image or PDF named after a fixture returns that fixture's text. The engine sits behind an interface, so a real OCR or VLM engine drops in without touching the service.

5 + 2
required endpoints, plus health and receipt status

3 / 3
fixtures match `gold.json` , in sync and async mode

84–98%
statement coverage per core package, all under `–race`

2
storage backends passing one 15-case contract suite

Run it

Command	What it does
<code>make run</code> (or <code>go run ./cmd/server</code>)	API on <code>:8080</code> , SQLite and uploads in <code>./data</code> . Needs Go 1.25+; <code>make docker</code> also works.
<code>make test</code>	Unit, gold, contract and end-to-end HTTP tests with the race detector; the HTTP suite runs on both backends.
<code>make demo</code>	With the server running, walks every endpoint over the three fixtures.

API

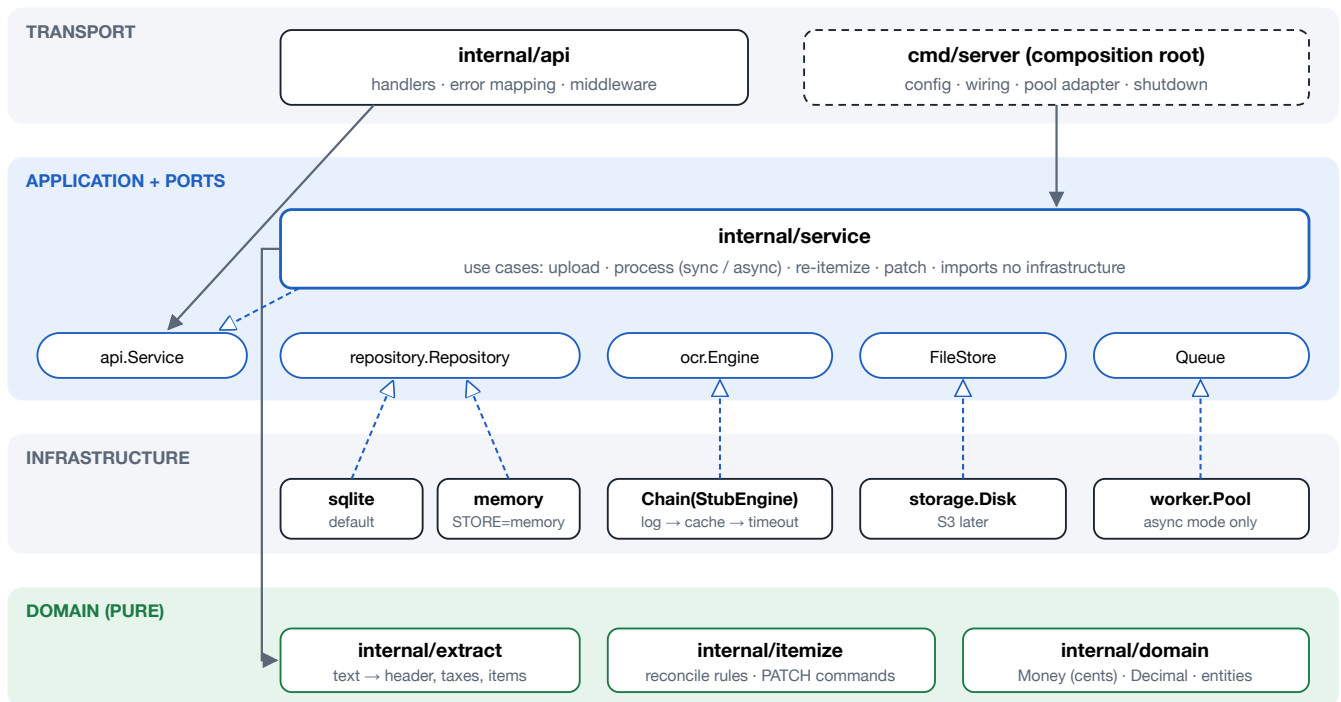
Endpoint	Behaviour
<code>POST /receipts</code>	Multipart upload. File type is sniffed from the bytes (JPEG, PNG, WebP, GIF, PDF, text). Returns <code>201</code> and <code>receipt_id</code> , and flags identical bytes as <code>duplicate_of</code> .
<code>POST /receipts/{id}/process</code>	OCR, extraction and auto-itemize. Creates or updates the one transaction. Sync by default (<code>200</code>); <code>PROCESS_MODE=async</code> answers <code>202</code> .
<code>GET /transactions/{id}</code>	Header, taxes, line items, <code>itemize_status</code> and issues. The version is exposed as the <code>ETag</code> .
<code>POST /transactions/{id}/itemize</code>	Re-runs auto-itemize from the stored OCR text. Replaces line items only; same transaction ID.
<code>PATCH /transactions/{id}/items</code>	Edit, merge, split, add, delete, applied all-or-nothing. <code>409</code> with the numbers if the result does not reconcile; optional <code>If-Match</code> (<code>412</code> when stale).
<code>GET /receipts/{id}</code> , <code>GET /health</code>	Receipt status (for async polling), OCR text and transaction link; liveness with a database ping.

Architecture

Receipt Auto-Itemize: Architecture

Navan take-home Task A

Dependencies point inward. The domain rules are pure functions, the service orchestrates use cases through interfaces, and infrastructure plugs in at the edges. `cmd/server` is the only place that knows which concrete pieces are used.



solid: uses · dashed: implements an interface declared where it is used

Why this shape

- **The service imports no infrastructure:** no HTTP, SQL, `os` or worker package. Uploads go through `FileStore`, OCR engines receive bytes, and the pool is wired in `cmd/server`.
- **Interfaces live where they are used:** `api.Service`, `service.FileStore` and `service.Queue`. The one exception is `repository`: two backends and the contract suite must import it without importing the service.
- **Domain rules are pure** (no I/O, clock or IDs), so they are tested directly against `gold.json`.

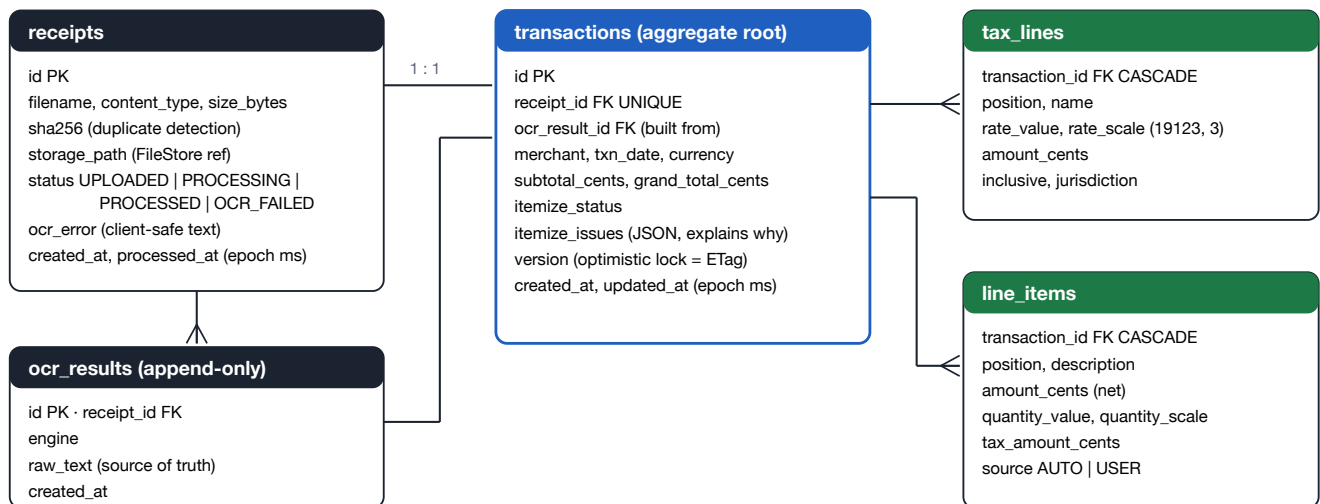
Extension points

- **A real OCR/VLM engine** implements `ocr.Engine` and inherits logging, caching and timeouts from the decorator chain.
- **Postgres** is a new package that passes `reptest.Run`, plus one case in `openRepository`.
- **S3** is a second `FileStore`. **A new PATCH op** is a new command type and one registry entry; `Apply` does not change.

Data model

Receipt Auto-Itemize: Architecture

Navan take-home Task A



Taxes are rows, not a header number

A receipt can carry several rates (7% food and 19% drinks). `inclusive` separates "VAT 19%" added on top from "incl. VAT 19%" already inside the prices; without it the taxi fixture could never reconcile.

Exact types, enforced

Money is `INTEGER` cents; instants are epoch milliseconds (UTC); rates and quantities are exact decimals stored as a value and a power of ten ($19.123\% = (19123, 3)$). Nothing is `REAL`, sums are overflow-checked, and tables are `STRICT`, so SQLite rejects a wrongly typed value.

Raw OCR is the source of truth

Every OCR run is kept. The transaction points at the run it was built from, so re-itemize is deterministic and re-processing keeps history.

Explainable status

`itemize_issues` carries the numbers behind `NEEDS_REVIEW` or `FAILED` (`TOTAL_MISMATCH`, `SUBTOTAL_MISMATCH`, `NO_ITEMS`, `NO_TOTAL`), so a review UI can say exactly what is wrong.

Results on the fixtures

Fixture	Taxes	Items	Status
receipt-clean	VAT 19%, 2.85, added on top	Espresso 3.50, Sandwich 8.90, Mineral water 2.60	COMPLETE
receipt-tax-only	VAT 19%, 3.83, inclusive	none ("Trip fare" has no price)	NEEDS_REVIEW NO_ITEMS
receipt-mismatch	VAT 19%, 1.90, added on top	Water 4.00, Snacks 6.00	NEEDS_REVIEW TOTAL_MISMATCH (11.90 vs 18.50)

Reconciliation rule

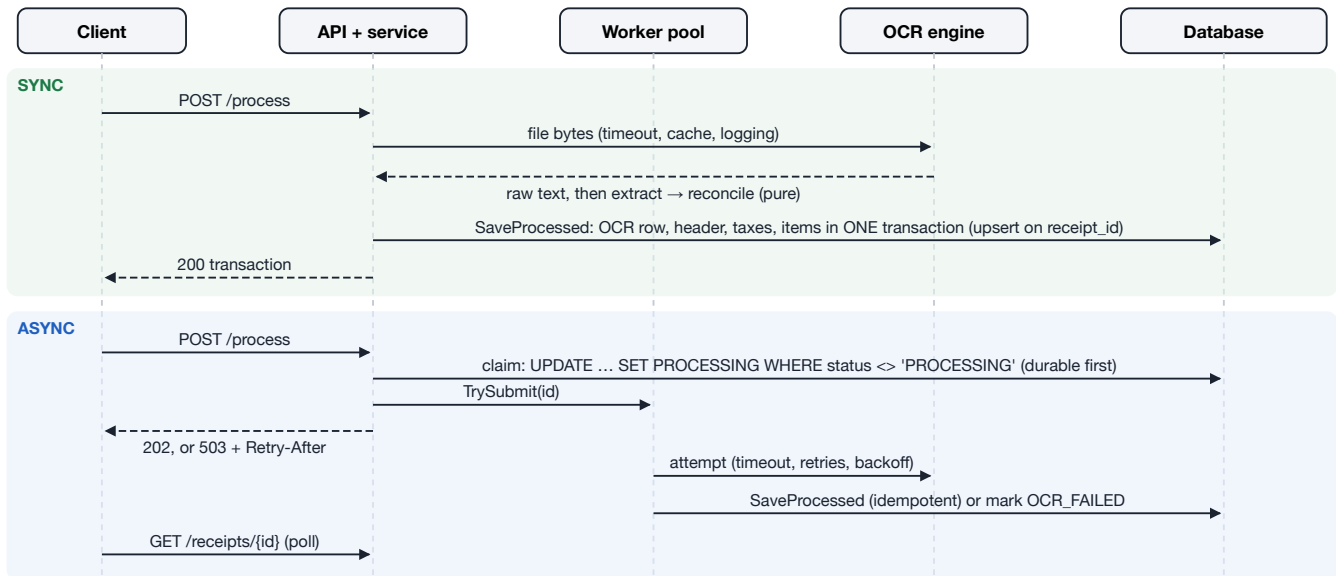
$\text{sum}(\text{items}) + \text{sum}(\text{non-inclusive taxes}) = \text{grand_total}$, within one cent. Items are net of added tax, which matches `gold.json`. Tip and rounding lines count toward the total but are left out of the printed-subtotal check. A mismatch is reported, never fixed: the service does not change the total and does not invent a balancing line.

Processing flows

Receipt Auto-Itemize: Architecture

Navan take-home Task A

Process: sync (default) and async (PROCESS_MODE=async)



Async guarantees

- **The database is the durable queue;** the channel is only a hand-off. A receipt is `PROCESSING` before it is queued, and on start every `PROCESSING` receipt is re-queued (snapshotted before the server accepts requests).
- **At-least-once, idempotent:** the save is an upsert on `receipt_id`, so a job that runs twice still yields one transaction.
- **Bounded:** `OCR_WORKERS` caps vendor concurrency; a full queue answers `503` and restores the receipt's previous state.

User edits (PATCH)

- Operations run in order on a copy; the first invalid one aborts with `422` and nothing is written.
- The result must reconcile with the stored total and taxes, or the answer is `409` with the issues and the proposed items.
- The write is `UPDATE ... WHERE version = ?`: a stale `If-Match` is `412`, and a lost race without one is `409`.
- Re-itemize and PATCH replace items only; header and taxes never change.

Code map

`cmd/server` config, wiring, graceful shutdown
`internal/service` use cases, interfaces only
`internal/itemize` reconcile rules, PATCH commands
`internal/repository` contract, `sqlite`, `memory`, `reptest`
`internal/storage` `FileStore` on local disk
`internal/id`, `internal/reqid` IDs; request ID in logs

`internal/api` handlers, error mapping, middleware
`internal/extract` OCR text → header, taxes, items
`internal/domain` Money, Rate, entities
`internal/ocr` engine, stub, decorators
`internal/worker` bounded pool, retries, shutdown
`fixtures/task-a` brief fixtures and `gold.json`

Invariants and failure handling

Receipt Auto-Itemize: Architecture

Navan take-home Task A

Invariant	Enforced by
One transaction per receipt, even under concurrent processing	<code>UNIQUE(receipt_id)</code> and <code>INSERT ... ON CONFLICT DO UPDATE</code>
Processing is atomic (OCR row, header, taxes, items)	<code>Repository.SaveProcessed</code> : one SQL transaction (SQLite) or one critical section (memory)
A transaction is read whole (its ETag matches its items)	Header, taxes and items are read inside one SQL transaction
Totals are never adjusted; no balancing line is invented	<code>itemize.Reconcile</code> only reports; PATCH answers <code>409</code> and writes nothing
Duplicate async <code>/process</code> calls run OCR once	Conditional <code>UPDATE ... WHERE status <> 'PROCESSING'</code> : exactly one caller sees a row change, on any database
No lost updates	<code>version</code> column, ETag / If-Match
Schema changes reach existing databases	Ordered, embedded migrations tracked in <code>PRAGMA user_version</code> ; a newer database is refused
File type is trusted from the bytes; stored names never come from client input	<code>http.DetectContentType</code> , an allow-list, and names built from the receipt ID

What happens when things fail

Failure	Behaviour
File has no readable text	Permanent: receipt marked <code>OCR_FAILED</code> with a client-safe reason; <code>422</code> in sync mode. Never retried.
OCR times out or vendor errors	Transient. Sync: <code>504 OCR_TIMEOUT</code> or <code>503 OCR_UNAVAILABLE</code> , receipt unchanged so the client can retry. Async: retried with capped exponential backoff. Both modes share one <code>Retryable</code> rule.
Client disconnects mid-request	Failure records and status restores run with <code>context.WithoutCancel</code> , so they still happen.
A job panics	Recovered in the worker and recorded as a permanent failure with its stack, so a poison receipt cannot crash-loop the process.
Process crashes or shuts down mid-job	Graceful shutdown stops HTTP, then drains workers within a deadline (backoff sleeps are abandoned). Unfinished receipts stay <code>PROCESSING</code> and are re-queued on start.
Handler panics	Middleware answers <code>500</code> in the standard error shape (unless the response has started) and the request is still access-logged; <code>http.ErrAbortHandler</code> propagates.
Internal error details	Never sent to clients: <code>ocr_error</code> and error bodies carry client-safe messages; the full error is logged with the request ID.

Every log line of a request carries its `request_id` (a context-aware `slog` handler), including OCR decorator logs. Callers may send `X-Request-ID` ; it is validated and echoed back.

Design patterns and SOLID

Receipt Auto-Itemize: Architecture

Navan take-home Task A

Pattern	Where	Why it is here
Repository	<code>internal/repository</code>	The service never sees a database; backends are swappable and prove it by passing one contract suite.
Strategy	<code>ocr.Engine</code> , <code>Repository</code> , <code>FileStore</code> , chosen in <code>cmd/server</code>	Stub OCR today, a vendor or VLM tomorrow; SQLite or memory; local disk or S3.
Decorator	<code>ocr.Chain</code> : logging, LRU cache by file hash, timeout	Cross-cutting concerns around the vendor call without touching the engine or the service.
Command	PATCH ops (<code>internal/itemize/ops.go</code>)	Each op is a type with <code>Execute</code> and its own accepted fields; a registry maps wire names, so a new op never grows <code>Apply</code> .
Adapter	<code>jobHandler</code> in <code>cmd/server</code>	Plugs the service's job methods into the generic pool; neither package imports the other.
Worker pool	<code>internal/worker</code>	Bounded concurrency, backpressure, retries, graceful shutdown.
Chain of responsibility	HTTP middleware	Request ID → access log → panic recovery around every route.
Value object	<code>domain.Money</code> , <code>domain.Rate</code>	Exact integer arithmetic; parsing and JSON in one place.
Optimistic locking	<code>version</code> , <code>ETag</code> / <code>If-Match</code>	No lost updates without holding locks.

Considered and left out: a state machine for receipt status (almost every transition is legal, since any receipt can be re-processed) and Specification objects for reconciliation (three short rules read better as one function).

S · O

Pure rules, orchestration, transport and storage are separate packages. New OCR engines, backends, file stores and PATCH ops are added, not edited in.

L · I

Both repositories are behaviourally identical (same contract suite). Small role interfaces: `Receipts`, `OCRResults`, `Transactions`, `FileStore`, `Queue`.

D

High-level policy depends on abstractions declared by the consumer; the composition root is the only place that names concrete types.

Testing and verification

Receipt Auto-Itemize: Architecture

Navan take-home Task A

Layer	What is tested
Domain (pure)	Extraction and reconciliation against <code>gold.json</code> ; PATCH commands; Money parsing, including a fuzz test for parse/print round-trips and overflow.
Contract suite	15 cases every backend must pass: round trips, one transaction per receipt, optimistic locking, atomic claims under concurrency, torn-read and sub-second ordering checks, ID collisions, isolation of returned values.
Service	Unit tests with fakes (file store, engine, queue, failure-injecting repository): upload cleanup, permanent vs transient OCR failures, queue-full restore, 409/412 semantics, safe failure messages.
Worker	Retries, backoff, timeouts, backpressure, shutdown drain and deadline, panics.
HTTP end to end	Every endpoint over real HTTP, gold results in sync and async mode, crash recovery, validation, error shape; the suite runs on both backends.

All tests run with the race detector. Beyond the test suite, the code was checked with `staticcheck`, `govulncheck` (no known vulnerabilities) and `golangci-lint`, and adversarially: bugs were only accepted when proven by a failing test, and each such test is kept as a regression test.

Statement coverage (`go test -cover`)

extract	98%	repository/memory	97%
worker	93%	domain	90%
itemize	90%	ocr	90%
api	89%	repository/sqlite	84%
service	84%	reqid	88%

Static analysis

- `go vet` and `gofmt`: clean (enforced by `make lint`).
- `staticcheck`: no findings.
- `govulncheck`: no known vulnerabilities.
- `golangci-lint` (17 linters incl. `gosec`, `errcheck`, `errorlint`): one real finding (upload directory permissions), fixed.

Adversarial testing

Four independent reviewers attacked concurrency, HTTP input, persistence and the domain rules. A bug only counted with a test that failed on the code. Examples of what was found and fixed:

Area	Bug, now covered by a regression test
Reconciliation	~185 maximum-size items made the int64 sum wrap onto the receipt total and save as COMPLETE; sums are now overflow-checked.
Extraction	"Total VAT" lines were read as the grand total; "19% VAT" (rate first) and German payment lines were booked as items; tips could never reconcile; a UTF-8 BOM hid the merchant.
Concurrency	A claim lease without a fencing token let a stale job run OCR again and overwrite a user's edits; the lease was removed (see tradeoffs).
Persistence	A torn read could pair version N with version N+1's items; the memory backend accepted a transaction-ID collision that SQLite refuses.
HTTP	A 300-byte file extension caused a 500; unmatched routes answered plain text instead of the JSON error shape.

Tradeoffs and the production path

Receipt Auto-Itemize: Architecture

Navan take-home Task A

Choice for this scope	Why	Production step
Stubbed OCR, regex parser over labelled text	The brief scores against fixtures; deterministic and testable.	A VLM behind <code>ocr.Engine</code> returning structured JSON with per-field confidence; low confidence maps to <code>NEEDS_REVIEW</code> , and the reconcile rules stay as the guardrail on model output.
SQLite, one connection	Zero setup; one writer avoids <code>SQLITE_BUSY</code> . It also serialises reads, which is this backend's ceiling.	Postgres as another <code>repository</code> package that passes <code>reptest</code> .
Local disk uploads	The brief allows it.	S3 with content-addressed keys as a second <code>FileStore</code> .
In-process worker pool	Real async semantics without extra infrastructure.	SQS or Kafka with separate workers; the job code is already idempotent and at-least-once.
Claims without a lease	A lease without a fencing token let a stale job overwrite newer results (found and proven in adversarial testing, then removed).	Lease plus a claim token carried with each job and checked on save.
Re-itemize and re-process replace user edits	The brief asks re-itemize to replace items; items carry <code>source</code> and clients may send <code>If-Match</code> .	<code>?preserve_user_edits=true</code> ; require <code>If-Match</code> on re-process once <code>USER</code> items exist.
No auth	Explicitly out of scope in the brief.	Auth and tenancy (<code>user_id</code> on every row), audit log of item edits.

Also next

- Allocate tax per line with largest-remainder rounding so line taxes sum exactly to the tax rows; support a rate per line.
- ISO 4217 currency exponents for zero- and three-decimal currencies (JPY, KWD); jurisdiction from the merchant's country.
- Duplicate detection beyond identical bytes (same merchant, date and total).
- Metrics (latency, error rate, `NEEDS_REVIEW` rate per OCR engine) and tracing on the request ID that is already propagated.